

EtherChannel

EtherChannel 是 Cisco 对链路聚合功能的常用名称。它可以把多条物理链路捆绑成一条逻辑链路，也就是 Port-Channel。

EtherChannel 的核心价值有三个：

- 提升链路总带宽。
- 提供链路冗余。
- 让 STP 把多条物理链路看成一条逻辑链路，避免并行链路被 STP 大量阻塞。

```
Gi1/0/23 + Gi1/0/24 => Port-channel1
```

为什么需要 EtherChannel

如果 SW1 和 SW2 之间连接两条普通 Trunk：

```
SW1 Gi1/0/23 ----- Gi1/0/23 SW2
SW1 Gi1/0/24 ----- Gi1/0/24 SW2
```

在二层网络中，这两条链路形成并行路径。STP 为了防止环路，通常会阻塞其中一条链路。

结果是：

- 有冗余。
- 正常情况下只有一条链路转发。
- 另一条链路处于阻塞状态，带宽没有被充分利用。

使用 EtherChannel 后：

```
SW1 Gi1/0/23 ----\
                        Po1 ----- Po1
SW1 Gi1/0/24 ----/
```

交换机会把多条物理链路当作一个 Port-Channel 逻辑接口处理。STP 看到的是 Port-Channel，而不是多条独立物理链路。

核心术语

术语	含义
Member Port	加入 EtherChannel 的物理接口
Channel Group	把物理接口绑定到某个聚合组的配置
Port-Channel	聚合后形成的逻辑接口
Bundle	成员接口成功加入聚合后的状态

术语	含义
LAG	Link Aggregation Group，链路聚合组的通用说法
LACPDU	LACP 使用的控制报文
PAgP Packet	PAgP 使用的控制报文

聚合方式

EtherChannel 常见有三种形成方式：

- 静态 on。
- PAgP。
- LACP。

静态 on

静态 on 不运行协商协议，两端都直接强制认为链路应加入 EtherChannel。

```
channel-group 1 mode on
```

特点：

- 不发送 LACPDU。
- 不发送 PAgP 报文。
- 两端必须都配置为 on。
- 不检查对端是否真的愿意聚合。
- 误配置时可能造成丢包或二层环路。

结论：

静态 on 可以理解，但实际网络中更常用 LACP。

PAgP

PAgP 的全称是 Port Aggregation Protocol，是 Cisco 私有协议，用于自动协商 EtherChannel。

PAgP 也属于 Cisco 私有二层控制协议，和前面 DTP/VTP 一样，可以通过 Cisco OUI + PID 识别。PAgP 常见 PID 为 0x0104。

PAgP 模式：

模式	含义
desirable	主动发送 PAgP，主动协商
auto	被动等待 PAgP 协商

PAgP 协商结果：

本端	对端	是否形成 EtherChannel
desirable	desirable	是
desirable	auto	是
auto	auto	否

记忆：

PAgP 的 desirable/auto 很像 DTP 的 desirable/auto：至少要有一端主动。

LACP

LACP 的全称是 Link Aggregation Control Protocol，是 IEEE 标准链路聚合控制协议。

LACP 不使用 Cisco 私有 PID。LACPDU 属于 IEEE Slow Protocols，常见识别点如下：

字段	常见值
Destination MAC	01:80:c2:00:00:02
EtherType	0x8809
Subtype	0x01

LACP 模式：

模式	含义
active	主动发送 LACPDU，主动协商
passive	被动等待 LACPDU

LACP 协商结果：

本端	对端	是否形成 EtherChannel
active	active	是
active	passive	是
passive	passive	否

结论：

多厂商互联时优先使用 LACP。常见稳定做法是两端都配置 active。

三种方式对比

方式	协议归属	模式	优点	风险/限制
Static on	无协商协议	on	简单直接	不检测对端，误配风险高
PAGP	Cisco 私有	desirable / auto	Cisco 环境可用	多厂商不通用
LACP	IEEE 802.1AX	active / passive	标准协议，多厂商通用	需要理解协商状态

成员口一致性要求

EtherChannel 能否形成，很大程度取决于成员接口配置是否一致。

二层 EtherChannel 常见要求：

- 速率一致。
- 双工一致。
- 接口类型兼容。
- 都是 Access，且 Access VLAN 一致。
- 或者都是 Trunk，且 Trunk 状态一致。
- Native VLAN 一致。
- Allowed VLAN 列表一致。
- STP 相关参数不能产生不兼容。

三层 EtherChannel 常见要求：

- 成员口都必须是三层接口。
- 成员口本身不配置 IP 地址。
- IP 地址配置在 Port-Channel 逻辑接口上。
- 成员口速率、双工、协商协议保持一致。

关键结论：

二层 EtherChannel 中，成员口必须都属于同一 VLAN，或者都作为 Trunk 工作。Native VLAN 不一致、Allowed VLAN 不一致、Access VLAN 不一致，都可能导致成员口无法加入聚合或被 suspend。

二层 EtherChannel

二层 EtherChannel 常用于交换机互联。

```
SW1 Gi1/0/23 ----- Gi1/0/23 SW2
SW1 Gi1/0/24 ----- Gi1/0/24 SW2

两条链路聚合为 Port-channel1
```

配置思路：

1. 物理接口加入同一个 channel-group。

2. 二层属性统一配置在 Port-Channel 上。
3. 成员物理口不要和 Port-Channel 上的二层属性互相冲突。

示例：

```
interface range gigabitEthernet 1/0/23 - 24
description To-SW2-EtherChannel
switchport
channel-group 1 mode active
no shutdown

interface port-channel 1
description To-SW2
switchport mode trunk
switchport trunk allowed vlan 10,20
switchport trunk native vlan 999
```

常见查看命令：

```
show etherchannel summary
show etherchannel 1 detail
show interfaces port-channel 1 switchport
show interfaces trunk
show lacp neighbor
show lacp counters
```

`show etherchannel summary` 中常见标记：

标记	含义
P	成员口已成功 bundled 到 Port-Channel
S	二层 Port-Channel
R	三层 Port-Channel
U	Port-Channel 正在使用
I	独立状态，未加入聚合
S	suspended
D	down
H	LACP hot-standby

示例输出：

```
Group  Port-channel  Protocol  Ports
-----+-----+-----+-----
1      Po1(SU)          LACP      Gi1/0/23(P) Gi1/0/24(P)
```

含义：

- `Po1(SU)`：二层 Port-Channel，正在使用。
- `LACP`：使用 LACP 协商。
- `Gi1/0/23(P)`：接口已成功加入聚合。

三层 EtherChannel

三层 EtherChannel 常用于汇聚到核心、核心互联、路由设备互联等场景。

```
Sw1 Po10 10.0.12.1/30 ----- 10.0.12.2/30 Po10 Sw2
```

配置思路：

1. 物理成员口执行 `no switchport`。
2. 物理成员口不配置 IP。
3. IP 配置在 Port-Channel 逻辑接口上。

示例：

```
interface range gigabitEthernet 1/0/47 - 48
description L3-EtherChannel-To-SW2
no switchport
no ip address
channel-group 10 mode active
no shutdown

interface port-channel 10
description L3-To-SW2
no switchport
ip address 10.0.12.1 255.255.255.252
no shutdown
```

常见查看命令：

```
show etherchannel summary
show ip interface brief
show interfaces port-channel 10
```

负载均衡原理

EtherChannel 的负载均衡不是把一个数据流平均切成多份，同时从多条链路发送。

它通常采用基于字段的哈希算法：

```
选择字段 -> 计算哈希 -> 得到数值 -> 映射到成员链路
```

同一条流的关键字段不变时，通常会被映射到同一条成员链路。多条不同流量分布到不同成员链路上，从整体上实现负载分担。

为什么不是逐包轮询

逐包轮询看起来很平均：

```
第 1 个包走 Gi1/0/23
第 2 个包走 Gi1/0/24
第 3 个包走 Gi1/0/23
第 4 个包走 Gi1/0/24
```

但这会带来乱序风险。

同一条 TCP 会话中，不同成员链路可能有不同队列、时延和拥塞状态。后发出的包可能先到，TCP 看到乱序后可能触发重复 ACK、重传、窗口收缩，最终导致性能下降。

因此 EtherChannel 通常按 flow 做哈希：

```
同一条流 -> 同一条成员链路
不同的流 -> 可能分散到不同成员链路
```

哈希计算的基本过程

可以把 EtherChannel 的负载分担理解成 4 步：

- 1. 设备根据全局负载均衡模式选择字段。
- 2. 从帧或包中取出这些字段。
- 3. 对字段做哈希或位运算，得到一个数值。
- 4. 把这个数值映射到 Port-Channel 的某条成员链路。

简化公式：

```
Hash(源字段，目的字段，可选端口字段) -> Hash Result -> Member Link
```

如果只有两条成员链路，可以粗略理解为：

```
Hash Result 落入 A 类 -> Gi1/0/23
Hash Result 落入 B 类 -> Gi1/0/24
```

真实设备的内部算法和桶数量由平台决定，不能简单等同于数学上的取模。但学习时可以先抓住本质：哈希结果决定成员链路。

哈希桶与链路数量

在一些 Catalyst 平台中，可以把哈希结果理解成 8 个桶。成员链路负责接收这些桶对应的流量。

当成员链路数刚好是 2、4、8 时，桶可以平均分配：

成员链路数	桶分布理解
8 条	1:1:1:1:1:1:1:1

成员链路数	桶分布理解
4 条	2:2:2:2
2 条	4:4

当成员链路数是 3、5、6、7 时，桶不容易完全均匀：

成员链路数	桶分布理解
7 条	2:1:1:1:1:1:1
6 条	2:2:1:1:1:1
5 条	2:2:2:1:1
3 条	3:3:2

结论：

2、4、8 条成员链路通常更容易得到相对均匀的哈希分布。3、5、6、7 条不是不能用，但要接受分布可能不完全平均。

总带宽与单流带宽

4 条 1G 链路聚合后，理论总带宽约为 4G：

```
Po1 = Gi1/0/21 + Gi1/0/22 + Gi1/0/23 + Gi1/0/24
```

但单个 TCP 流通常仍然只走其中一条成员链路：

单个大文件传输 -> 可能只有 1G 上限
多个会话并发 -> 才可能接近 4G 总吞吐

结论：

EtherChannel 是统计意义上的负载分担，不是每一条流都能使用聚合后的总带宽。

常见负载均衡字段

不同 Catalyst 平台支持的选项不完全相同，最终以设备命令为准：

```
port-channel load-balance ?
```

常见字段：

类型	常见算法	使用字段	适合场景
二层	src-mac	源 MAC	多个源主机访问少量目的时可能有用
二层	dst-mac	目的 MAC	少量源访问多个目的时可能有用

类型	常见算法	使用字段	适合场景
二层	src-dst-mac	源 MAC + 目的 MAC	二层交换场景常用，分布通常比单字段更好
三层	src-ip	源 IP	多源访问同一目的时可考虑
三层	dst-ip	目的 IP	单源访问多目的时可考虑
三层	src-dst-ip	源 IP + 目的 IP	三层网络中常用，通常比单字段更均衡
四层	src-port	源 TCP/UDP 端口	平台支持时，可用于大量会话场景
四层	dst-port	目的 TCP/UDP 端口	业务端口差异明显时可考虑
四层	src-dst-port	源端口 + 目的端口	大量会话、东西向流量多时可能更均衡
混合	IP + L4 Port	IP 地址和四层端口组合	数据中心、服务器多会话流量更容易均衡

选择原则：

使用变化最丰富、最能区分流量的字段。字段越能区分不同业务流，就越容易把流量分散到不同成员链路。

负载均衡场景

多 PC 访问同一个网关



如果按目的 MAC：

- 目的 MAC 都是网关 MAC。
- 分布可能较差。

如果按源 MAC：

- PC1、PC2、PC3 的源 MAC 不同。
- 分布可能更好。

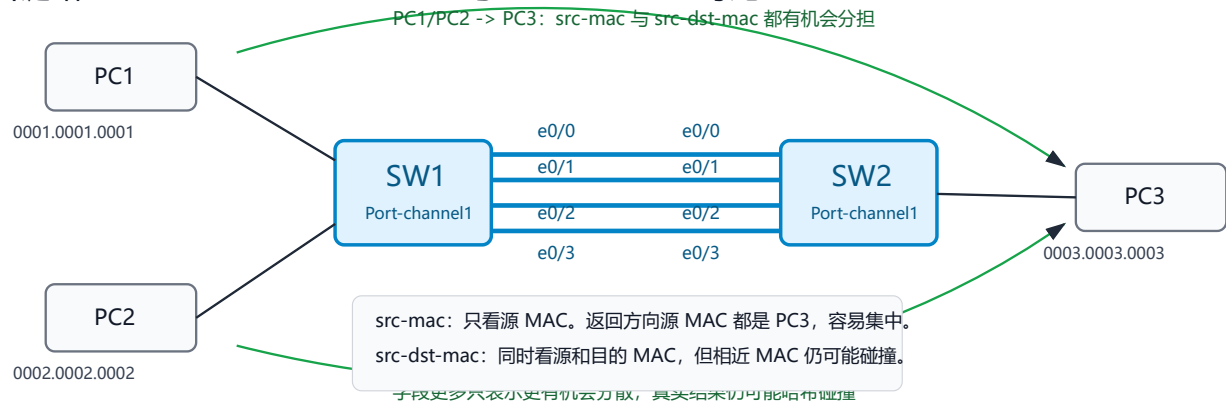
适合考虑：

```
port-channel load-balance src-mac
port-channel load-balance src-dst-mac
```

四条链路下比较 src-mac 与 src-dst-mac

下面拓扑中，SW1 和 SW2 之间使用 4 条物理链路组成一个 EtherChannel。

四链路 EtherChannel: src-mac 与 src-dst-mac 对比



```
PC1 ----\
          SW1 == Po1: e0/0,e0/1,e0/2,e0/3 == SW2 ---- PC3
PC2 ----/
```

假设 MAC 地址如下:

设备	MAC 地址
PC1	0001.0001.0001
PC2	0002.0002.0002
PC3	0003.0003.0003

流量方向一: PC1、PC2 访问 PC3。

流量	源 MAC	目的 MAC
PC1 -> PC3	0001.0001.0001	0003.0003.0003
PC2 -> PC3	0002.0002.0002	0003.0003.0003

如果 SW1 使用 `src-mac`:

```
Hash(PC1 源 MAC) -> 某条成员链路
Hash(PC2 源 MAC) -> 某条成员链路
```

因为 PC1 和 PC2 的源 MAC 不同, SW1 往 SW2 方向可能把两条流量分到不同成员链路。

如果 SW1 使用 `src-dst-mac`:

```
Hash(PC1 源 MAC, PC3 目的 MAC) -> 某条成员链路
Hash(PC2 源 MAC, PC3 目的 MAC) -> 某条成员链路
```

因为两条流量的 MAC 对不同, 也可能分到不同成员链路。

只看 PC1/PC2 到 PC3 的方向, `src-mac` 和 `src-dst-mac` 都有机会产生分担。

真正的差异通常出现在返回方向。

流量方向二：PC3 返回 PC1、PC2。

流量	源 MAC	目的 MAC
PC3 -> PC1	0003.0003.0003	0001.0001.0001
PC3 -> PC2	0003.0003.0003	0002.0002.0002

如果 SW2 使用 `src-mac`：

```
Hash(PC3 源 MAC) -> 某条成员链路
Hash(PC3 源 MAC) -> 同一组关键字段
```

两条返回流量的源 MAC 都是 PC3，因此哈希输入字段相同，很容易落到同一条成员链路上。

如果 SW2 使用 `src-dst-mac`：

```
Hash(PC3 源 MAC, PC1 目的 MAC) -> 某条成员链路
Hash(PC3 源 MAC, PC2 目的 MAC) -> 某条成员链路
```

两条返回流量的目的 MAC 不同，哈希输入字段不同，更有机会分散到不同成员链路。

对比结论：

算法	PC1/PC2 -> PC3	PC3 -> PC1/PC2	特点
<code>src-mac</code>	多个源 MAC，可能分担	同一个源 MAC，容易集中	适合“多源往外走”的单向观察
<code>src-dst-mac</code>	MAC 对不同，可能分担	MAC 对不同，也可能分担	双向通信通常更均衡

注意：

- EtherChannel 的哈希是本地行为，SW1 和 SW2 各自独立计算。
- 同一条流通常只走一条成员链路。
- 4 条成员链路不代表 2 条流一定能占满 4 条链路。
- 要看到 4 条链路都产生明显流量，需要更多不同的源/目的组合，或者更多不同会话。
- 真实映射结果由平台哈希算法决定，不能只靠肉眼计算 MAC 地址推断。

示例配置：

```
! 只看源 MAC
port-channel load-balance src-mac

! 同时看源 MAC 和目的 MAC
port-channel load-balance src-dst-mac
```

如果平台不支持该命令，可以用连续流量配合成员接口计数器观察：

```
show interfaces e0/0 | include packets|rate
show interfaces e0/1 | include packets|rate
show interfaces e0/2 | include packets|rate
show interfaces e0/3 | include packets|rate
```

实测：src-dst-ip 与 src-dst-mac 的差异

下面是一组四成员 EtherChannel 的实测结果。

地址信息：

设备	IP 地址	MAC 地址
PC1	192.168.1.1	00:50:79:66:68:21
PC2	192.168.1.2	00:50:79:66:68:22
PC3	192.168.1.3	00:50:79:66:68:23

成员链路：

```
sw1 e0/0 --- e0/0 sw2
sw1 e0/1 --- e0/1 sw2
sw1 e0/2 --- e0/2 sw2
sw1 e0/3 --- e0/3 sw2
```

实测结果：

负载均衡方式	流量	实际成员链路
src-dst-ip	PC1 ping PC3	e0/2
src-dst-ip	PC2 ping PC3	e0/1
src-dst-mac	PC1 ping PC3	e0/0
src-dst-mac	PC2 ping PC3	e0/0

这个结果说明：

- src-dst-ip 在这组 IP 地址下能把 PC1 和 PC2 的流量分到不同链路。
- src-dst-mac 虽然同时看源 MAC 和目的 MAC，但这两组 MAC 对在当前平台的哈希结果发生了碰撞，都落到了 e0/0。
- src-dst-mac 的字段更多，不代表每一组流量一定分散。

用 XOR 理解 src-dst-ip

很多平台的 EtherChannel 哈希可以用“选择字段后做 XOR 或类似位运算，再映射到内部桶”来理解。

对 src-dst-ip 来说，这组地址非常适合观察低位变化。

PC1 到 PC3：

```
源 IP: 192.168.1.1 = c0.a8.01.01
目 IP: 192.168.1.3 = c0.a8.01.03
```

```
XOR:
c0.a8.01.01
c0.a8.01.03
-----
00.00.00.02
```

如果用 4 条成员链路做简化管理，可以看结果低 2 bit：

```
0x02 = binary 10
binary 10 -> 成员链路 2 -> e0/2
```

PC2 到 PC3：

```
源 IP: 192.168.1.2 = c0.a8.01.02
目 IP: 192.168.1.3 = c0.a8.01.03

XOR:
c0.a8.01.02
c0.a8.01.03
-----
00.00.00.01
```

低 2 bit：

```
0x01 = binary 01
binary 01 -> 成员链路 1 -> e0/1
```

所以这组 `src-dst-ip` 的实测结果很容易解释：

```
PC1 -> PC3: XOR 结果低位是 10 -> e0/2
PC2 -> PC3: XOR 结果低位是 01 -> e0/1
```

为什么 src-dst-mac 没有分散

如果只做最简单的 MAC XOR：

PC1 到 PC3：

```
源 MAC: 00:50:79:66:68:21
目 MAC: 00:50:79:66:68:23

XOR = 00:00:00:00:00:02
```

PC2 到 PC3：

```
源 MAC: 00:50:79:66:68:22
目 MAC: 00:50:79:66:68:23

XOR = 00:00:00:00:00:01
```

如果真实算法只是“拿 MAC XOR 结果的最后 2 bit 直接对应 4 条链路”，那么两条流量应该分别落到不同链路。

但实测结果是：

```
PC1 -> PC3: e0/0
PC2 -> PC3: e0/0
```

这说明当前平台的 `src-dst-mac` 哈希不是简单地把“最后 2 bit 的 XOR 结果”直接映射到成员口。它可能还会做进一步折叠、选择特定位置、映射到内部哈希桶，再由哈希桶映射到成员链路。

可以把真实过程理解成：

```
取源/目的 MAC
-> 平台内部哈希或 XOR-like 计算
-> 得到内部 hash bucket
-> bucket 映射到某个成员链路
```

因此，不同的源/目的 MAC 对仍然可能得到相同 bucket，这就是哈希碰撞。

这组实验的价值在于：

EtherChannel 负载均衡字段越丰富，通常越有机会分散流量，但永远不能保证每两条流都落到不同成员链路。真实结果要看平台哈希算法和实际字段组合。

观察 EtherChannel 分担时，通过清空接口计数器后持续产生流量，再比较成员接口的 packet 计数增量：

```
clear counters e0/0
clear counters e0/1
clear counters e0/2
clear counters e0/3

show interfaces e0/0 | include packets|rate
show interfaces e0/1 | include packets|rate
show interfaces e0/2 | include packets|rate
show interfaces e0/3 | include packets|rate
```

三层 Port-Channel

```
Distribution SW ---- L3 Port-Channel ---- Core SW
```

三层互联中，IP 字段通常比 MAC 字段更能反映业务差异。

常见选择：

```
port-channel load-balance src-dst-ip
```

原因：

- 源 IP 和目的 IP 组合通常比单独源或目的更分散。
- 适合多网段、多业务互访。
- 比只看 MAC 更贴近三层转发流量特征。

同一对 IP 之间大量会话

```
Client 10.1.1.10 -> Server 10.2.2.20:443
Client 10.1.1.10 -> Server 10.2.2.20:8443
Client 10.1.1.10 -> Server 10.2.2.20:22
```

如果只按 `src-dst-ip`：

- 源 IP 相同。
- 目的 IP 相同。
- 多个会话可能都走同一条成员链路。

如果平台支持 IP + 四层端口字段，可能更容易分散：

```
port-channel load-balance src-dst-port
```

或使用平台支持的扩展负载均衡选项。

多服务器东西向流量

```
Server A ----\
Server B ----- Tor SW ----- Po1 ----- Core / Spine
Server C ----/
```

如果业务流量源 IP、目的 IP、端口变化都比较丰富，使用 IP + L4 port 类型的算法更容易获得均衡效果。

如果平台只支持 MAC 或 IP，则通常优先考虑 `src-dst-ip`。

查看与配置负载均衡

查看当前负载均衡算法：

```
show etherchannel load-balance
```

示例输出：

```
EtherChannel Load-Balancing Configuration:
      src-dst-ip
```

配置负载均衡算法：

```
configure terminal
port-channel load-balance src-dst-ip
end
```

注意：

- `port-channel load-balance` 通常是全局配置，会影响设备上的 EtherChannel。
- 不同平台支持的算法不同。
- 修改算法可能改变流量到成员链路的映射，生产环境中应谨慎变更。

验证某条流可能走哪条成员链路：

```
test etherchannel load-balance interface port-channel 1 ip 10.1.1.10 10.2.2.20
```

不同平台的 `test etherchannel load-balance` 参数可能不同，以 ? 显示为准。

查看成员链路实际流量：

```
show interfaces gigabitEthernet 1/0/23
show interfaces gigabitEthernet 1/0/24
show interfaces port-channel 1
```

重点看：

- input rate。
- output rate。
- packets input/output。
- errors。
- drops。

排障思路

1. 看汇总状态

```
show etherchannel summary
```

重点看：

- Port-Channel 是否为 `U`。
- 成员口是否为 `P`。
- 是否出现 `I`、`S`、`D`。
- Protocol 是 LACP、PAgP 还是静态。

2. 看协议邻居

LACP:

```
show lacp neighbor
show lacp counters
```

PAgP:

```
show pagp neighbor
show pagp counters
```

3. 看配置一致性

```
show running-config interface range gigabitEthernet 1/0/23 - 24
show running-config interface port-channel 1
show interfaces port-channel 1 switchport
show interfaces trunk
```

重点看:

- 速率、双工是否一致。
- 二层/三层模式是否一致。
- Access VLAN 是否一致。
- Trunk Native VLAN 是否一致。
- Trunk allowed VLAN 是否一致。
- Channel-group 编号和模式是否一致。

4. 看日志

```
show logging | include EC|LACP|PAgP|suspended|bundle|channel
```

常见日志方向:

- 成员口配置不一致。
- Native VLAN 不一致。
- 对端未启用 LACP。
- passive + passive。
- 一端 static on, 另一端 LACP/PAgP。

常见故障

现象	常见原因	排查方向
Port-Channel 不起来	两端模式不匹配	查 <code>show etherchannel summary</code> 和配置

现象	常见原因	排查方向
成员口 <code>I</code>	没有成功加入聚合，独立运行	查 LACP/PAgP 邻居和配置一致性
成员口 <code>s</code>	接口被 suspended	查速率、双工、VLAN、Trunk、日志
LACP passive + passive 不起来	双方都不主动	至少一端改 active，常用 active + active
PAgP auto + auto 不起来	双方都不主动	至少一端改 desirable
单流没有叠加带宽	哈希按流分担，不拆单流	用多会话测试，查看负载均衡算法
某条成员链路流量明显偏高	哈希字段与业务流量不匹配，或成员数导致桶不均	查看当前算法、业务模型和接口计数
Trunk VLAN 不通	Port-Channel 上 allowed VLAN 或 native VLAN 配置有问题	查 <code>show interfaces trunk</code>

小结

- EtherChannel 把多条物理链路聚合成一条 Port-Channel 逻辑链路。
- LACP 是 IEEE 802.1AX 标准协议，PAgP 是 Cisco 私有协议。
- LACP 常用 `active + active`，PAgP 至少要有一端 `desirable`。
- 成员口配置必须一致，否则成员口可能无法加入聚合或被 suspend。
- 二层 EtherChannel 的二层属性通常配置在 Port-Channel 上。
- 三层 EtherChannel 的 IP 地址配置在 Port-Channel 上，成员物理口不配置 IP。
- EtherChannel 负载均衡通常按哈希，不是把单个流逐包平均拆分。
- 哈希字段应根据业务流量模型选择。
- 2、4、8 条成员链路通常更容易获得均匀桶分布。
- 排障先看 `show etherchannel summary`，再看 LACP/PAgP 邻居、配置一致性和日志。